

HANOI UNIVERSITY OF SCIENCE AND
TECHNOLOGY
School of Information and communications technology

Test Plan
Version 1.0

AIMS Project
Subject: ITSS Software
Development

Group 23

Bùi Trung Hiếu - 20235932
Nguyễn Hữu Tuấn Tú - 20236005
Nguyễn Tuấn Khanh - 20235952
Nguyễn Trần Bình - 20235899
Trần Trung Nghĩa - 20235983

Hanoi, May, 2026

Table of Contents

- 1 [Introduction](#)
- 2 [Overall Description](#)
- 3 [Testing Approach Strategy'](#)
- 4 ['Unit Testing Summary'!A1](#)
- 5 ['Test Case Details'!A1](#)

1. Introduction

1.1. Objective

This Unit Testing Plan document was prepared for AIMS Group 23 to define the objectives, scope, and testing strategy for the Unit Testing phase of the project. This document is intended for developers to ensure the accuracy of individual software components such as product management functions, shipping cost calculation, and payment integration (PayPal, VietQR). This document will be developed in parallel with the project's source code.

1.2 Scope

Our project focuses on developing the AIMS product management and e-commerce system. The scope of this Unit Test Plan covers our core modules:

- Product Management (adding, editing, deleting, and adjusting inventory)
- Order Processing (creating orders and calculating shipping fees)
- Payment Integration through PayPal Sandbox and VietQR.

This phase excludes actual database or UI integration testing, as we will be using mocks. Our primary goal is to detect bugs early and ensure the business logic is entirely accurate.

1.3. Glossary

- UT (Unit Testcase): Testing of the smallest testable parts of an application.
- UC (Use Case): A description of a system's behavior as it responds to a request from a user.
- TDD (Test-Driven Development): A software development process relying on software requirements being converted to test cases before software is fully developed.
- Mock: Simulated objects that mimic the behavior of real objects in controlled ways to isolate the unit under test.
- Traceability Matrix: A document that co-relates test cases to functional requirements.

1.4. References

- Jest Documentation. Retrieved from: <https://jestjs.io/>

- NGUYEN Thi Thu Trang. AIMS: An Internet Media Store - CAPSTONE-PROJECT
PROBLEM STATEMENT
- NGUYEN Thi Thu Trang. Lab 10: Unit Testing - ITSS SOFTWARE
DEVELOPMENT. SOICT - HUST

2. Overall Description

2.1. General Overview

AIMS is an e-commerce platform for purchasing physical media, providing a seamless shopping experience for customers and inventory control tools for managers. Regarding its development history, the project has completed the SRS and SDD phases and is currently in the implementation phase, utilizing Test-Driven Development (TDD) to verify core business logic. High-level diagrams (Use Case, Activity, and Package diagrams) remain as defined in the previous SRS and SDD documents. However, the Sequence Diagrams have been updated. Specifically, a new external actor (lifeline) was added to represent the third-party payment service (PayPal Sandbox). This change was made to accurately reflect the independent communication with external payment gateways and to establish a solid basis for mocking the payment module during this Unit Testing phase.

2.2. Assumptions/Constraints/Risks

2.2.1. Assumptions

Development Process: The team strictly follow Test-Driven Development (TDD) strategy, consistently following the Red-Green-Refactor cycle.

Input Data: The product specifications (Books, CDs, DVDs, and Newspapers) and shipping fee calculation rules (for Hanoi, HCMC, and other provinces) detailed in the Problem Statement are accurate and will remain unchanged throughout the unit testing phase.

Mock Environment: We do test under the assumption that all external dependencies, such as the database and third-party APIs (PayPal, VietQR), can be fully substituted with mocks or stubs. This guarantees the independence of our unit tests.

Tools: Jest has been successfully integrated into the project.

2.2.2. Constraints

- **Testing Technique:** We must utilize Black-box testing, focusing strictly on input and output based on business requirements, without interfering with the internal source code structure.
- **Unit Scope:** Unit tests will exclusively target individual modules and methods (such as `validateDeliveryInfo` and `calculateShippingFee`). They should not be used to test complex user interfaces or interactions between large subsystems.
- **Language & Framework:** We will be using Jest for this project.
- **Directory Structure:** When submitting code to GitHub, the required directory structure must be strictly followed, ensuring that the test directory is located within the `api` folder (e.g., `api/test`).

2.2.3. Risks

Business Logic Risks: The complexity of our shipping fee rules (such as a maximum 25k VND discount for orders exceeding 100k VND, and weight-based pricing at 0.5kg or 3kg) increases the risk of missing boundary values. **Solution:** Design test cases that specifically target these threshold points (100,000 VND, 3kg, and 0.5kg).

Mock Integration Risks: Mocking our payment gateways (PayPal/VietQR) may not perfectly replicate the actual response logic of the Sandbox environment. **Solution:** Carefully review the official API documentation and implement additional test cases dedicated to handling exceptions and error codes.

Maintenance Risks: Rapid changes to the source code can cause older unit tests to fail (Regression). **Solution:** Enforce running the entire test suite after every code refactor.

3. Testing Approach/Strategy

3.1. Overview

- Unit Testing Strategy: Our approach focuses on verifying the individual components of the AIMS application, such as services, controllers, and functions.
- Testing Technique: We are exclusively applying a Black-box testing strategy for this project. Test cases are designed solely around the functional requirements and expected behaviors outlined in the problem statement, without examining the internal source code. This approach ensures that the system accurately fulfills the business logic and user needs.
- Dependency Isolation: All external dependencies, including the database and APIs, will be strictly isolated using Mocks and Stubs via Jest.

3.2. Scope of Unit Testing

Unit Testing Scope: The scope encompasses the functions, methods, services, and controllers responsible for the core business logic of the AIMS system.

In-Scope Components:

- Product Management:
 - Product.validatePrice: Validating the 30% - 150% base price constraints.
 - ProductController.delete: Ensuring products are only deleted when out of stock, and enforcing limits (10 items per bulk action, 20 items daily quota).
 - Product.adjustStock: Validating manual stock updates (requiring a reason, ensuring

inputs are non-negative integers).

- ProductDetailMapper: Mapping detailed attributes for specific categories (Books, Newspapers, CDs, DVDs).
 - Order & Delivery:
 - DeliveryInfoValidator: Validating receiver names (letters only), phone numbers (10 digits with separator handling), and capping addresses/delivery instructions at 200 characters.
 - ShippingFeeService.calculateShippingFee: Testing region-based logic (Hanoi/HCMC vs. other provinces), weight tiers (0.5kg/3kg), and shipping discounts (max 25k free for orders > 100k).
 - Payment Integration:
 - createOrder, capturePayment, refundPayment: Handling PayPal Sandbox API flows, including decimal formatting and error handling for tokens/APIs.
 - VietqrService: Testing Access Token retrieval, QR code generation, and Callback logic (digital signature validation, amount matching, and duplicate transaction prevention).
 - rejectPaidVietQROrder: Handling manual refund notifications for rejected VietQR orders.
 - Product Services:
 - ProductDetailService & ProductAvailabilityService: Verifying product detail viewing permissions, product ID validation, and stock statuses (Available, Out of stock, Unavailable).
 - CartQuantityValidator: Ensuring cart additions do not exceed actual inventory.
- Out-of-Scope Components:
- End-to-end (E2E) UI testing.
 - Direct interactions with the actual database (to be replaced entirely by Mocks/Stubs).
 - Real network connections to PayPal/VietQR payment gateways (utilizing Sandbox environments and Mock Axios instead).
 - Full system workflow testing across multiple integrated modules.

3.3. Strategy for Designing Unit Tests

Our test cases are designed entirely using the following Black-box techniques:

- Equivalence Partitioning: We categorize input data into valid and invalid groups to ensure comprehensive coverage (for example, validating phone number formats or checking for positive versus negative product quantities).
- Boundary Value Analysis: We focus heavily on critical thresholds, such as the 100,000 VND order mark, the 0.5kg and 3kg weight tiers, and product pricing limits (ranging from 30% to 150% of the original price).
- Decision Tables: We apply this method to handle complex business logic, such as determining whether to approve or reject an order based on a combination of inventory levels and payment status.

3.4. Tools and Frameworks

JEST: Used as the primary testing framework to automate unit tests.

Mocking: Utilized Jest's mocking features to isolate modules from external

dependencies.

4. Unit Testing Summary

4.1. Traceability from Test Cases to Use Cases

<Mark the cell with an "x" where the Test Case tests.>

USE CASE ID			UC234.1 Add Product	UC234.2 Edit Product	UC234.3 Adjust Quantity	UC234.4 Delete Product	UC233 Pay by Credit Card	UC232 Pay by VietQR	UC235 View Product	UC231 Place Order
Test Case ID	Test CaseTitle	Totals	8	8	4	4	13	10	19	16
UT_PM_001	validatePrice - Price at lower bound (30%)	2	x	x						
UT_PM_002	validatePrice - Price below minimum bound	2	x	x						
UT_PM_003	validatePrice - Price at upper bound (150%)	2	x	x						
UT_PM_004	validatePrice - Price exceeds maximum bound	2	x	x						
UT_PM_005	validatePrice - Price is zero or negative	2	x	x						
UT_PM_006	validatePrice - Valid price level (Middle)	2	x	x						
UT_PM_007	validatePrice - Price data is non-numeric	2	x	x						
UT_PM_008	deleteProduct - Delete product when stock = 0	1				x				
UT_PM_009	deleteProduct - Delete product while in stock	1				x				
UT_PM_010	deleteProduct - Bulk delete exceeds limit	1				x				
UT_PM_011	deleteProduct - Exceeds daily deletion limit	1				x				
UT_PM_012	adjustStock - Update stock with reason	1			x					
UT_PM_013	adjustStock - Update stock without reason	1			x					
UT_PM_014	adjustStock - Decrease stock causing negative quantity	1			x					
UT_PM_015	adjustStock - Adjustment quantity is non-integer	1			x					
UT_PM_016	validateBook - Validate Book information (Author, Publisher)	2	x	x						
UT_PM_017	validateDVD - Validate DVD information (Director, Studio)	2	x	x						
UT_ORD_001	createOrder - Successfully calls API with valid amount	1					x			
UT_ORD_002	createOrder - Handles decimal formatting (toFixed)	1					x			
UT_ORD_003	createOrder - Handles error when getAccessToken fails	1					x			
UT_ORD_004	createOrder - Handles API error responses (400/500)	1					x			
UT_PAY_001	capturePayment - Valid transaction	1					x			
UT_PAY_002	capturePayment - Insufficient card balance	1					x			
UT_PAY_003	capturePayment - Duplicate payment	1					x			
UT_REF_001	refundPayment - Full refund	1					x			
UT_REF_002	refundPayment - Valid partial refund	1					x			
UT_REF_003	refundPayment - Refund exceeds captured amount	1					x			
UT_REF_004	refundPayment - Refund for unpaid order	1					x			
UT_VQR_001	Get access token successfully	1					x	x		
UT_VQR_002	Get access token failed	1					x	x		
UT_VQR_003	Generate QR code successfully	1						x		
UT_VQR_004	Generate QR code failed due to VietQR API error	1						x		
UT_VQR_005	Generate QR code failed due to invalid amount	1						x		
UT_VQR_006	Handle valid payment callback successfully	1						x		
UT_VQR_007	Reject callback with invalid signature	1						x		
UT_VQR_008	Reject callback when amount does not match order total	1						x		
UT_VQR_009	Prevent duplicate transaction from duplicate callback	1						x		
UT_VQR_010	Notify manual refund required for rejected VietQR order	1						x		
UT_PO_001	validateReceiverName - Reject blank receiver name	1								x
UT_PO_002	validateReceiverName - Reject receiver name containing digits	1								x
UT_PO_003	validatePhoneNumber - Accept phone number with exactly 10 digits	1								x
UT_PO_004	validatePhoneNumber - Accept phone number using one separator type	1								x
UT_PO_005	validatePhoneNumber - Reject phone number using mixed separators	1								x
UT_PO_006	validatePhoneNumber - Reject phone number with fewer than 10 digits	1								x
UT_PO_007	validateAddress - Reject blank delivery address	1								x
UT_PO_008	validateShippingInstructions - Accept empty shipping instructions	1								x
UT_PO_009	validateShippingInstructions - Accept shipping instructions at 200-character boundary	1								x
UT_PO_010	validateShippingInstructions - Reject shipping instructions over 200 characters	1								x
UT_PO_011	calculateShippingFee - Calculate base shipping fee for Hanoi/HCMC within first 3kg	1								x
UT_PO_012	calculateShippingFee - Calculate base shipping fee for other provinces	1								x
UT_PO_013	calculateShippingFee - Calculate extra weight fee for other provinces	1								x
UT_PO_014	calculateShippingFee - Calculate extra weight fee for Hanoi/HCMC orders over 3kg	1								x
UT_PO_015	calculateShippingFee - Apply free-shipping discount when subtotal exceeds 100,000 VND	1								x
UT_PO_016	calculateShippingFee - Limit free-shipping discount to maximum 25,000 VND	1								x
UT_VPD_001	validateProductid - Accept valid product identifier	1							x	
UT_VPD_002	validateProductid - Reject blank or null product identifier	1							x	
UT_VPD_003	validateProductid - Reject invalid product identifier format	1							x	

UT_VPD_004	findProductById - Return product detail when product exists	1								x	
UT_VPD_005	findProductById - Return not found when product does not exist	1								x	
UT_VPD_006	canViewProductDetail - Allow customer without login	1								x	
UT_VPD_007	canViewProductDetail - Reject unauthenticated product manager	1								x	
UT_VPD_008	getAvailabilityStatus - Active product with stock is available	1								x	
UT_VPD_009	getAvailabilityStatus - Active product with zero stock is out of stock	1								x	
UT_VPD_010	getAvailabilityStatus - Deactivated product is unavailable	1								x	
UT_VPD_011	mapGeneralProductDetail - Map common product fields	1								x	
UT_VPD_012	mapBookDetail - Map book-specific fields	1								x	
UT_VPD_013	mapNewspaperDetail - Map newspaper-specific fields	1								x	
UT_VPD_014	mapCdDetail - Map CD-specific fields and track list	1								x	
UT_VPD_015	mapDvdDetail - Map DVD-specific fields	1								x	
UT_VPD_016	validateAddToCartQuantity - Accept quantity within stock	1								x	
UT_VPD_017	validateAddToCartQuantity - Reject quantity exceeding stock	1								x	
UT_VPD_018	getAvailabilityStatus - Active product with zero or negative stock is out of stock	1								x	
UT_VPD_019	mapProductDetail - Route to correct mapper based on product type casing	1								x	

4.2. Test Suite for UC234.1-"Add Product" and UC234.2-"Edit Product"

Test Suite ID	Test Suite Title	Description	Test Cases
US001	Price Rule Suite	Testing all business rule related	UT_PM_1,...,UT_PM_007
US002	Validate Addition Information	Testing all business rule related	UT_PM_016, UT_PM_017

4.2. Test Suite for UC234.3-"Adjust Quantity"

Test Suite ID	Test Suite Title	Description	Test Cases
US003	Reason Rule Suite	Testing all business rule related	UT_PM_12,UT_PM_015

4.2. Test Suite for UC234.4-"Delete Product"

Test Suite ID	Test Suite Title	Description	Test Cases
US004	Product Quantity Delete Rule Suite	Testing all business rule related	UT_PM_008,...,UT_PM_00
US005	Product Delete Limitation Rule Suite	Testing all business rule related	UT_PM_010, UT_PM_011

4.2. Test Suite for UC233 -"Pay order by Credit Card"

Test Suite ID	Test Suite Title	Description	Test Cases
US006	PayPal API Integration Suite	Testing communication flow,	UT_ORD_001,
US007	Payment Capture Suite	Testing all business rules related	UT_PAY_001,
US008	Refund Process Suite	Testing all business rules related	UT_REF_001, UT_REF_002,

4.2. Test Suite for UC232 -"Pay order by VietQR"

Test Suite ID	Test Suite Title	Description	Test Cases
US009	VietQR Payment Suite	Verify the VietQR payment flow,	UT_VQR_001,

4.2. Test Suite for UC231-"Place Order"

Test Suite ID	Test Suite Title	Description	Test Cases
US010	Delivery Information Validation	Unit tests for receiver name,	UT_PO_001,
US011	Shipping Fee Calculation	Unit tests for Hanoi/HCMC and	UT_PO_011,

4.2. Test Suite for UC231-"View Product Detail"

Test Suite ID	Test Suite Title	Description	Test Cases
US012	Product Detail Validation and Retrieval Suite	Unit tests for validating product	UT_VPD_001, UT_VPD_002,
US013	Product Detail Access and Availability Suite	Unit tests for checking	UT_VPD_006, UT_VPD_007,
US014	Product Detail Display and Cart Action Suite	Unit tests for mapping	UT_VPD_011, UT_VPD_012,

5. Test Case Details

Test Case ID	Test Case Name	Description	Unit Under Test	Input Data	Expected Output	Actual Output	Pass/Fail	Notes
UT_PM_001	Validate Lower Price	Check price at the 30% minimum limit	`Product.validatePrice(orig, curr)`	`orig=200.000`, `curr=60.000`	`TRUE`	TRUE	pass	
UT_PM_002	Test Price Below Min	Check price below the 30% limit	`Product.validatePrice(orig, curr)`	`orig=200.000`, `curr=59.000`	`FALSE`	FALSE	pass	
UT_PM_003	Validate Upper Price	Check price at the 150% maximum limit	`Product.validatePrice(orig, curr)`	`orig=100.000`, `curr=150.000`	`TRUE`	TRUE	pass	
UT_PM_004	Test Price Above Max	Check price exceeding the 150% limit	`Product.validatePrice(orig, curr)`	`orig=100.000`, `curr=150.100`	`FALSE`	FALSE	pass	
UT_PM_005	Price Invalid Input	Enter selling price as 0 or negative	`Product.validatePrice(orig, curr)`	`curr=-10.000`	`FALSE` (Invalid Price)	FALSE	pass	
UT_PM_006	Price Middle Valid	Check price within the allowed range	`Product.validatePrice(orig, curr)`	`orig=100.000`, `curr=80.000`	`TRUE`	TRUE	pass	
UT_PM_007	Price Type Mismatch	Enter alphabetic characters into the price field	`Product.validatePrice(orig, curr)`	`curr="abc"`	`FALSE` (Numeric Required)	FALSE	pass	
UT_PM_008	Delete with Zero Stock	Delete product when out of stock	`ProductController.delete(product)`	`stock=0`	`Status: DELETED`	{ status: 'DELETED' }	pass	
UT_PM_009	Delete with In-Stock	Delete product while still in stock	`ProductController.delete(product)`	`stock=10`	`Status: DEACTIVATED`	{ status: 'DEACTIVATED' }	pass	
UT_PM_010	Bulk Delete Limit	Check single deletion limit	`ProductController.delete(list)`	`List size = 11`	`Throw Exception`	Throw BadRequestException	pass	
UT_PM_011	Daily Delete Quota	Check daily deletion limit	`ProductController.delete(product)`	`21st delete in day`	`Throw Exception`	Throw HttpException	pass	
UT_PM_012	Stock Update Reason	Update inventory with a reason	`Product.adjustStock(qty, reason)`	`qty=-5`, `reason="Hong"`	`TRUE` (Updated)	TRUE	pass	
UT_PM_013	Missing Stock Reason	Update inventory without a reason	`Product.adjustStock(qty, reason)`	`qty=10`, `reason=""`	`FALSE` (Error)	FALSE	pass	
UT_PM_014	Negative Stock Result	ease inventory resulting in negative :	`Product.adjustStock(qty, reason)`	`curr=5`, `adj=-10`	`FALSE` (Stock >= 0)	FALSE	pass	
UT_PM_015	Stock Non-Integer	Enter inventory quantity as a decima	`Product.adjustStock(qty, reason)`	`qty=1.5`	`FALSE` (Integer required)	FALSE	pass	
UT_PM_016	Validate Book Info	Check book information	`Book.validate()`	`Type:Book`, `Author:null`	`FALSE`	FALSE	pass	
UT_PM_017	Validate DVD Info	Check DVD information	`DVD.validate()`	`Type:DVD`, `Director:null`	`FALSE`	FALSE	pass	
UT_ORD_001	createOrder - Valid API call	Verify successful API call to PayPal Sandbox returns order data	createOrder(amount)	amount=100.00	Return { id: "ORD123" }	Return { id: "ORD123" }		Valid conditions met
				Mock Axios: status: 201, data: { id: "ORD123" }				pass
UT_ORD_002	createOrder - Decimal formatting	Ensure the amount is automatically formatted to exactly 2 decimal places	createOrder(amount)	amount=99.9	Axios called with payload value: "99.90"	value: "99.90"	pass	Validates toFixed(2) logic
UT_ORD_003	createOrder - Auth token failure	Handle error execution flow when retrieving Access Token fails	createOrder(amount)	amount=50.00	Throw Error	Throw Error Error: "Failed to get access token"		Auth layer protection
				Mock getAccessToken: Throw Error	Axios post not called			pass
UT_ORD_004	createOrder - API rejection	Handle 4xx/5xx errors returned from PayPal Order API	createOrder(amount)	amount=150.00	Throw Error containing Axios response details	Throw Error		Ensures catch block works
				Mock Axios: Throw AxiosError(400)				pass
UT_PAY_001	capturePayment - Valid capture	Verify successful payment capture API call updates transaction	capturePayment(orderId)	orderId="ORD123"	Return { status: "COMPLETED" }	Return { status: "COMPLETED" }		Valid conditions met
UT_PAY_002	capturePayment - Capture declined	Handle payment declined by external payment gateway	capturePayment(orderId)	Mock Axios: status: 201, data: { status: "COMPLETED" }	Throw Error with failure reason	Throw Error		Gateway rejection handling
				orderId="ORD123"				pass
UT_PAY_003	capturePayment - Already captured	Handle error when capturing an order state that is already paid	capturePayment(orderId)	orderId="ORD_PAID"	Throw Error preventing duplicate charge	Throw Error		Prevent double charging
				Mock Axios: Throw AxiosError(400)				pass

UT_REF_001	refundPayment - Full refund	Verify successful full refund API call execution	refundPayment(captureId, amount)	captureId="CAP123", amount=100.00 Mock Axios: status: 201	Return { status: "COMPLETED" }	Return { status: "COMPLETED" }	pass	Valid conditions met
UT_REF_002	refundPayment - Partial refund	Verify successful partial refund API call execution	refundPayment(captureId, amount)	captureId="CAP123", amount=50.00 Mock Axios: status: 201	Return { status: "COMPLETED" }	Return { status: "COMPLETED" }	pass	Tests payload logic
UT_REF_003	refundPayment - Exceed amount	Handle refund amount exceeding the initially captured amount	refundPayment(captureId, amount)	amount=200.00 (exceeds captured) Mock Axios: Throw AxiosError(422)	Throw Error	Throw Error	pass	Sandbox rejects over-refund
UT_REF_004	refundPayment - Unsettled transaction	Handle error when trying to refund a transaction not yet settled	refundPayment(captureId, amount)	captureId="CAP123" Mock Axios: Throw AxiosError(400)	Throw Error indicating invalid status	Throw Error	pass	Timing edge case handling
UT_PO_001	validateReceiverName - Reject blank receiver name	Verify that receiver name is mandatory and cannot be blank.	DeliveryInfoValidator.validateReceiverName(receiverName)	receiverName = ""	Return validation error: Receiver name is required.	Validation error returned: Receiver name is required.	Pass	Negative case for required delivery field.
UT_PO_002	validateReceiverName - Reject receiver name containing digits	Verify that receiver name accepts letters only and rejects numeric characters.	DeliveryInfoValidator.validateReceiverName(receiverName)	receiverName = 'Nguyen Van A1'	Return validation error: Receiver name must contain letters only.	Validation error returned: Receiver name must contain letters only.	Pass	Equivalence partitioning: invalid name format.
UT_PO_003	validatePhoneNumber - Accept phone number with exactly 10 digits	Verify that a phone number with exactly 10 digits is accepted.	DeliveryInfoValidator.validatePhoneNumber(phoneNumber)	phoneNumber = '0981413168'	Return TRUE / valid phone number.	TRUE returned; phone number is valid.	Pass	Boundary value: exactly 10 digits.
UT_PO_004	validatePhoneNumber - Accept phone number using one separator type	Verify that a phone number with separators is accepted when only one separator type is used.	DeliveryInfoValidator.validatePhoneNumber(phoneNumber)	phoneNumber = '0981 413 168'	Return TRUE / valid phone number after normalizing separators.	TRUE returned; phone number is valid after normalizing separators.	Pass	Valid equivalence class: one separator type.
UT_PO_005	validatePhoneNumber - Reject phone number using mixed separators	Verify that a phone number is rejected when it mixes separator types.	DeliveryInfoValidator.validatePhoneNumber(phoneNumber)	phoneNumber = '0981 413-168'	Return validation error: Phone number must use only one separator type.	Validation error returned: Phone number must use only one separator type.	Pass	Invalid equivalence class: mixed separators.
UT_PO_006	validatePhoneNumber - Reject phone number with fewer than 10 digits	Verify that a phone number with fewer than 10 digits is rejected.	DeliveryInfoValidator.validatePhoneNumber(phoneNumber)	phoneNumber = '098141316'	Return validation error: Phone number must contain exactly 10 digits.	Validation error returned: Phone number must contain exactly 10 digits.	Pass	Boundary value: 9 digits.
UT_PO_007	validateAddress - Reject blank delivery address	Verify that delivery address is mandatory and cannot be blank.	DeliveryInfoValidator.validateAddress(address)	address = ' '	Return validation error: Delivery address is required.	Validation error returned: Delivery address is required.	Pass	Negative case for required delivery field.
UT_PO_008	validateShippingInstructions - Accept empty shipping instructions	Verify that shipping instructions are optional and may be empty.	DeliveryInfoValidator.validateShippingInstructions(shippingInstructions)	shippingInstructions = ""	Return TRUE / valid shipping instructions.	TRUE returned; shipping instructions are valid.	Pass	Optional-field valid equivalence class.
UT_PO_009	validateShippingInstructions - Accept shipping instructions at 200-character boundary	Verify that shipping instructions with exactly 200 characters are accepted.	DeliveryInfoValidator.validateShippingInstructions(shippingInstructions)	shippingInstructions = 'A' repeated 200 times	Return TRUE / valid shipping instructions.	TRUE returned; shipping instructions are valid.	Pass	Boundary value: maximum valid length.
UT_PO_010	validateShippingInstructions - Reject shipping instructions over 200 characters	Verify that shipping instructions longer than 200 characters are rejected.	DeliveryInfoValidator.validateShippingInstructions(shippingInstructions)	shippingInstructions = 'A' repeated 201 times	Return validation error: Shipping instructions must not exceed 200 characters.	Validation error returned: Shipping instructions must not exceed 200 characters.	Pass	Boundary value: just above maximum length.
UT_PO_011	calculateShippingFee - Calculate base shipping fee for Hanoi/HCMC within first 3kg	Verify that orders delivered to Hanoi or HCMC use the 22,000 VND base fee for the first 3kg.	ShippingFeeService.calculateShippingFee(province, totalWeightKg, subtotalBeforeVAT)	province = 'Hanoi'; totalWeightKg = 3.0; subtotalBeforeVAT = 90,000	Return shippingFee = 22,000 VND.	Shipping fee returned: 22,000 VND.	Pass	Major-city base-fee rule.
UT_PO_012	calculateShippingFee - Calculate base shipping fee for other provinces	Verify that orders delivered outside Hanoi/HCMC use the 30,000 VND base fee for the first 0.5kg.	ShippingFeeService.calculateShippingFee(province, totalWeightKg, subtotalBeforeVAT)	province = 'Da Nang'; totalWeightKg = 0.5; subtotalBeforeVAT = 90,000	Return shippingFee = 30,000 VND.	Shipping fee returned: 30,000 VND.	Pass	Other-province base-fee rule.
UT_PO_013	calculateShippingFee - Calculate extra weight fee for other provinces	Verify that other-province orders add 2,500 VND for each subsequent 0.5kg after the first 0.5kg.	ShippingFeeService.calculateShippingFee(province, totalWeightKg, subtotalBeforeVAT)	province = 'Da Nang'; totalWeightKg = 1.0; subtotalBeforeVAT = 90,000	Return shippingFee = 32,500 VND (30,000 + 2,500).	Shipping fee returned: 32,500 VND.	Pass	Boundary value: one extra 0.5kg block.
UT_PO_014	calculateShippingFee - Calculate extra weight fee for Hanoi/HCMC orders over 3kg	Verify that Hanoi/HCMC orders add 2,500 VND for each subsequent 0.5kg after the first 3kg.	ShippingFeeService.calculateShippingFee(province, totalWeightKg, subtotalBeforeVAT)	province = 'Ho Chi Minh City'; totalWeightKg = 3.5; subtotalBeforeVAT = 90,000	Return shippingFee = 24,500 VND (22,000 + 2,500).	Shipping fee returned: 24,500 VND.	Pass	Boundary value: one extra 0.5kg block after 3kg.

UT_PO_015	calculateShippingFee - Apply free-shipping discount when subtotal exceeds 100,000 VND	Verify that a shipping discount is applied when product subtotal exceeds 100,000 VND.	ShippingFeeService.calculateShippingFee(province, totalWeightKg, subtotalBeforeVAT)	province = 'Hanoi'; totalWeightKg = 3.0; subtotalBeforeVAT = 120,000	Return finalShippingFee = 0 VND because 22,000 VND is fully discounted.	Final shipping fee returned: 0 VND because the shipping fee is fully discounted.	pass	Decision table: subtotal > 100,000 VND and raw fee <= 25,000 VND.
UT_PO_016	calculateShippingFee - Limit free-shipping discount to maximum 25,000 VND	Verify that the free-shipping discount cannot exceed 25,000 VND per order.	ShippingFeeService.calculateShippingFee(province, totalWeightKg, subtotalBeforeVAT)	province = 'Da Nang'; totalWeightKg = 6.0; subtotalBeforeVAT = 200,000	Return finalShippingFee = 32,500 VND (raw fee 57,500 - discount cap 25,000).	Final shipping fee returned: 32,500 VND after applying the 25,000 VND discount cap.	pass	Decision table: subtotal > 100,000 VND and raw fee > 25,000 VND.
UT_VQR_001	getAccessToken - Get access token successfully	Verify that the system can get a valid VietQR access token when valid clientId and apiKey are provided.	VietqrService.getAccessToken()	clientId="valid-client-id"; apiKey="valid-api-key"; Mock VietQR token API returns accessToken="token_123".	Return a non-empty valid access token.	Return a non-empty valid access token.	pass	Authentication success case.
UT_VQR_002	getAccessToken - Get access token failed	Verify that the system handles authentication failure when VietQR credentials are invalid.	VietqrService.getAccessToken()	clientId="invalid-client-id"; apiKey="invalid-api-key"; Mock VietQR token API returns 401 Unauthorized.	Throw/return authentication error and do not save access token.	Return/throw authentication error and no token is saved.	pass	Negative case for invalid credentials.
UT_VQR_003	generateQRCode - Generate QR code successfully	Verify that the system generates QR code data successfully with valid order information.	VietqrService.generateQRCode(orderId, amount, content)	orderId="ORDER001"; amount=250000; content="PAY ORDER001"; valid access token; Mock VietQR QR API returns QRCodeDataDto.	Return QR code data including qrCode, qrContent, amount and order information.	Return QR code data including qrCode, qrContent, amount and order information.	pass	Main flow: display QR code for customer.
UT_VQR_004	generateQRCode - Handle VietQR API error	Verify that the system handles error response when VietQR API fails to generate QR code.	VietqrService.generateQRCode(orderId, amount, content)	orderId="ORDER001"; amount=250000; content="PAY ORDER001"; Mock VietQR QR API returns 500/Error.	Throw/return QR generation error and no QR code is displayed to customer.	Return/throw QR generation error and no QR code is displayed.	pass	External service error handling.
UT_VQR_005	generateQRCode - Reject invalid amount	Verify that the system rejects QR generation when payment amount is invalid.	VietqrService.generateQRCode(orderId, amount, content)	orderId="ORDER001"; amount=0; content="PAY ORDER001".	Reject the request and throw/return invalid amount error.	Reject the request and return invalid amount error.	pass	Input validation before calling VietQR API.
UT_VQR_006	handleCallback - Valid payment callback	Verify that a valid successful VietQR callback is accepted and transaction information is saved.	VietqrController.handleCallback(callbackDto) / VietqrService.handleCallback(callbackDto)	orderId="ORDER001"; transactionId="TXN001"; amount=250000; status="SUCCESS"; signature="valid-signature"; order total=250000.	Save payment transaction; mark order as paid; move order to pending processing state.	Save payment transaction, mark order as paid, and move order to pending processing state.	pass	Callback success flow.
UT_VQR_007	handleCallback - Reject invalid signature	Verify that callback with invalid signature is rejected and no transaction is saved.	VietqrController.handleCallback(callbackDto) / VietqrService.handleCallback(callbackDto)	orderId="ORDER001"; transactionId="TXN001"; amount=250000; status="SUCCESS"; signature="invalid-signature".	Reject callback; do not save transaction; do not update order payment status.	Reject callback, do not save transaction, and do not update payment status.	pass	Security test against forged callback.
UT_VQR_008	handleCallback - Reject amount mismatch	Verify that payment callback is rejected if paid amount is different from order total.	VietqrController.handleCallback(callbackDto) / VietqrService.handleCallback(callbackDto)	orderId="ORDER001"; orderTotal=250000; callbackAmount=200000; transactionId="TXN001"; signature="valid-signature".	Reject callback because paid amount does not match order total; transaction is not saved as successful.	Reject callback because paid amount does not match order total; transaction is not saved as successful.	pass	Prevents incorrect payment confirmation.
UT_VQR_009	handleCallback - Prevent duplicate transaction	Verify that the system does not create duplicate transaction when the same callback is received more than once.	VietqrController.handleCallback(callbackDto) / VietqrService.handleCallback(callbackDto)	orderId="ORDER001"; transactionId="TXN001"; amount=250000; existing transaction TXN001 already saved.	Do not create duplicate transaction; keep order status unchanged.	Do not create duplicate transaction and keep order status unchanged.	pass	Idempotency / duplicate webhook handling.
UT_VQR_010	rejectPaidVietQROrder - Notify manual refund required	Verify that the system notifies Product Manager to refund manually when a paid VietQR order is rejected.	PaymentService.rejectPaidVietQROrder(orderId, rejectReason) / OrderService.rejectOrder(orderId, rejectReason)	orderId="ORDER001"; paymentMethod="VIETQR"; orderStatus="PAID"; rejectReason="Out of stock".	Notify Product Manager that manual refund is required; do not trigger automatic refund API.	Notify Product Manager that manual refund is required and do not trigger automatic refund API.	pass	VietQR does not support automatic refund in current scope.
UT_VPD_001	validateProductId - Accept Valid ID	Verify that a valid product identifier is accepted before retrieving product detail.	ProductDetailValidator.validateProductId(productId)	productId = 'BK10293847'	Return TRUE / valid product id.	TRUE	pass	Covers valid partition: non-empty alphanumeric barcode/product id.
UT_VPD_002	validateProductId - Reject Blank ID	Verify that blank or null product identifier is rejected.	ProductDetailValidator.validateProductId(productId)	productId = null; productId = ""	Return FALSE or validation error: Product id is required.	FALSE	pass	Covers boundary/invalid partition: null and empty string.
UT_VPD_003	validateProductId - Reject Invalid Format	Verify that product identifier with unsupported characters is rejected.	ProductDetailValidator.validateProductId(productId)	productId = './bad-id' or 'BK1029@3847'	Return FALSE or validation error: Product id format is invalid.	FALSE	pass	Black-box test for invalid format input.

UT_VPD_004	findProductById - Product Exists	Verify that the method returns product detail data when the selected product exists in catalog.	ProductDetailsService.findProductById(product Id)	productId = 'BK10293847'; mocked repository returns product object	Return product object/detail DTO containing product name, type, barcode, price, description, weight, and dimensions.	Return product detail DTO with product name, type, barcode, price, description, weight and dimensions.	pass	Covers main flow: selected product exists.
UT_VPD_005	findProductById - Product Not Found	Verify that the method handles a valid product id that does not exist in catalog.	ProductDetailsService.findProductById(product Id)	productId = 'UNKNOWN_001'; mocked repository returns null	Return ProductNotFound result or throw NotFoundException; no detail DTO is returned.	Return ProductNotFound result	pass	Covers alternative flow: product not found.
UT_VPD_006	canViewProductDetail - Customer Allowed	Verify that a customer can view product detail without login, as customer login is not required.	ProductDetailAccessChecker.canViewProductDetail(actor, product)	actor = { role: 'Customer', isAuthenticated: false }; product exists	Return TRUE / access allowed.	TRUE	pass	Based on AIMS requirement that customers do not need login.
UT_VPD_007	canViewProductDetail - Product Manager Unauthorized	Verify that product manager detail access is rejected if the manager is not authenticated.	ProductDetailAccessChecker.canViewProductDetail(actor, product)	actor = { role: 'ProductManager', isAuthenticated: false }; product exists	Return FALSE or Unauthorized/Forbidden result.	FALSE	pass	Covers role/access rule for Product Manager.
UT_VPD_008	getAvailabilityStatus - Active and In Stock	Verify availability status for an active product with positive stock.	ProductAvailabilityService.getAvailabilityStatus(status, stock, actor)	status = 'ACTIVE'; stock = 12; actor.role = 'Customer'	Return status 'Available' and canAddToCart = TRUE.	Return status 'Available' and canAddToCart = TRUE	pass	Valid partition for purchasable product.
UT_VPD_009	getAvailabilityStatus - Zero Stock	Verify availability status for an active product with zero stock.	ProductAvailabilityService.getAvailabilityStatus(status, stock, actor)	status = 'ACTIVE'; stock = 0; actor.role = 'Customer'	Return status 'Out of stock' and canAddToCart = FALSE.	Return status 'Out of stock' and canAddToCart = FALSE.	pass	Boundary value: stock = 0.
UT_VPD_010	getAvailabilityStatus - Deactivated Product	Verify that a deactivated product is shown as unavailable for sale.	ProductAvailabilityService.getAvailabilityStatus(status, stock, actor)	status = 'DEACTIVATED'; stock = 5; actor.role = 'Customer'	Return availability notice 'Unavailable for sale' and canAddToCart = FALSE.	Return availability notice 'Unavailable for sale' and canAddToCart = FALSE.	pass	Covers UC235 alternative flow for unavailable product.
UT_VPD_011	mapGeneralProductDetail - Common Fields	Verify that common product fields are mapped correctly to the detail response.	ProductDetailMapper.mapGeneralProductDetail(product)	product = { name:'Doraemon Volume 1', type:'Book', price:85000, originalValue:70000, barcode:'BK10293847', description:'Comic book', weight:350, dimensions:'20 x 14 x 2 cm' }	Return DTO containing product name, type, selling price, original value, barcode, description, weight with unit, and dimensions.	Return DTO containing product name, type, selling price, original value, barcode, description, weight with unit, and dimensions.	pass	Covers output general data of product detail.
UT_VPD_012	mapBookDetail - Book Specific Fields	Verify that book-specific information is mapped correctly.	ProductDetailMapper.mapBookDetail(book)	book = { authors:['Fujiko F. Fujio'], coverType:'Paperback', publisher:'Kim Dong', publicationDate:'2024-01-01' }	Return Book detail fields: authors, cover type, publisher, and publication date.	Return book detail fields: authors, cover type, publisher, and publication date.	pass	Product type partition: Book.
UT_VPD_013	mapNewspaperDetail - Newspaper Specific Fields	Verify that newspaper-specific information is mapped correctly.	ProductDetailMapper.mapNewspaperDetail(newspaper)	newspaper = { editorInChief:'Nguyen Van A', publisher:'HUST News', publicationDate:'2026-03-16', issn:'1234-5678' }	Return Newspaper detail fields: editor-in-chief, publisher, publication date, and ISSN when available.	Return newspaper detail fields: editor-in-chief, publisher, publication date, and ISSN.	pass	Product type partition: Newspaper.
UT_VPD_014	mapCdDetail - CD Specific Fields	Verify that CD-specific information and track list are mapped correctly.	ProductDetailMapper.mapCdDetail(cd)	cd = { artists:['Artist A'], recordLabel:'Label A', genre:'Pop', tracks:[{title:'Track 1', length:'03:30'}] }	Return CD detail fields: artists, record label, genre, and track list with title and length.	Return CD detail fields: artists, record label, genre, and track list with title and length.	pass	Product type partition: CD.
UT_VPD_015	mapDvdDetail - DVD Specific Fields	Verify that DVD-specific information is mapped correctly.	ProductDetailMapper.mapDvdDetail(dvd)	dvd = { discType:'Blu-ray', director:'Christopher Nolan', runtime:120, studio:'Studio A', language:'English', subtitles:['Vietnamese'] }	Return DVD detail fields: disc type, director, runtime, studio, language, and subtitles.	Return DVD detail fields: disc type, director, runtime, studio, language, and subtitles.	pass	Product type partition: DVD.
UT_VPD_016	validateAddToCartQuantity - Valid Quantity	Verify that quantity is accepted when customer adds an available product from detail page within stock.	CartQuantityValidator.validate(quantity, stock, productStatus)	quantity = 2; stock = 12; productStatus = 'ACTIVE'	Return TRUE / valid quantity.	TRUE	pass	Related to AIMS rule that customers can add product from detail screen.
UT_VPD_017	validateAddToCartQuantity - Quantity Exceeds Stock	Verify that quantity greater than stock is rejected before adding product to cart.	CartQuantityValidator.validate(quantity, stock, productStatus)	quantity = 13; stock = 12; productStatus = 'ACTIVE'	Return FALSE or validation error: Quantity exceeds available stock.	FALSE	pass	Boundary/invalid partition for add-to-cart quantity.
UT_VPD_018	Get Availability Status - Zero Or Negative Stock	If the product is listed as ACTIVE but the stock is 0 or negative, it means it's out of stock.	ProductAvailabilityService.getAvailabilityStatus(product, actor)	status="ACTIVE", stock=0/-1, role="Customer"	status="Out of stock", canAddToCart=false	Return status "Out of stock" and canAddToCart = FALSE.	pass	Added after review feedback